Scraping Service Analysis

Date No.

Abstact: This report explains the architecture and design of a scraping service that collects book data from books.toscrape.com, cleans and stores the data in SQLite, exposes it through a FastAPI interface, runs as a production-ready Linux service, and can be packaged as a .deb installable.

Problem Statement:- The system must:

1. scrape product data from a target Website.
2. Clean and normalize fields for Consistent Storage.
3. Persist data in a database.
4. expose query endpoints through a REST API.
5. run as a managed Linux Service.
6. be distributable as a Debian Package.

Date No.

End-to-End Data Flow

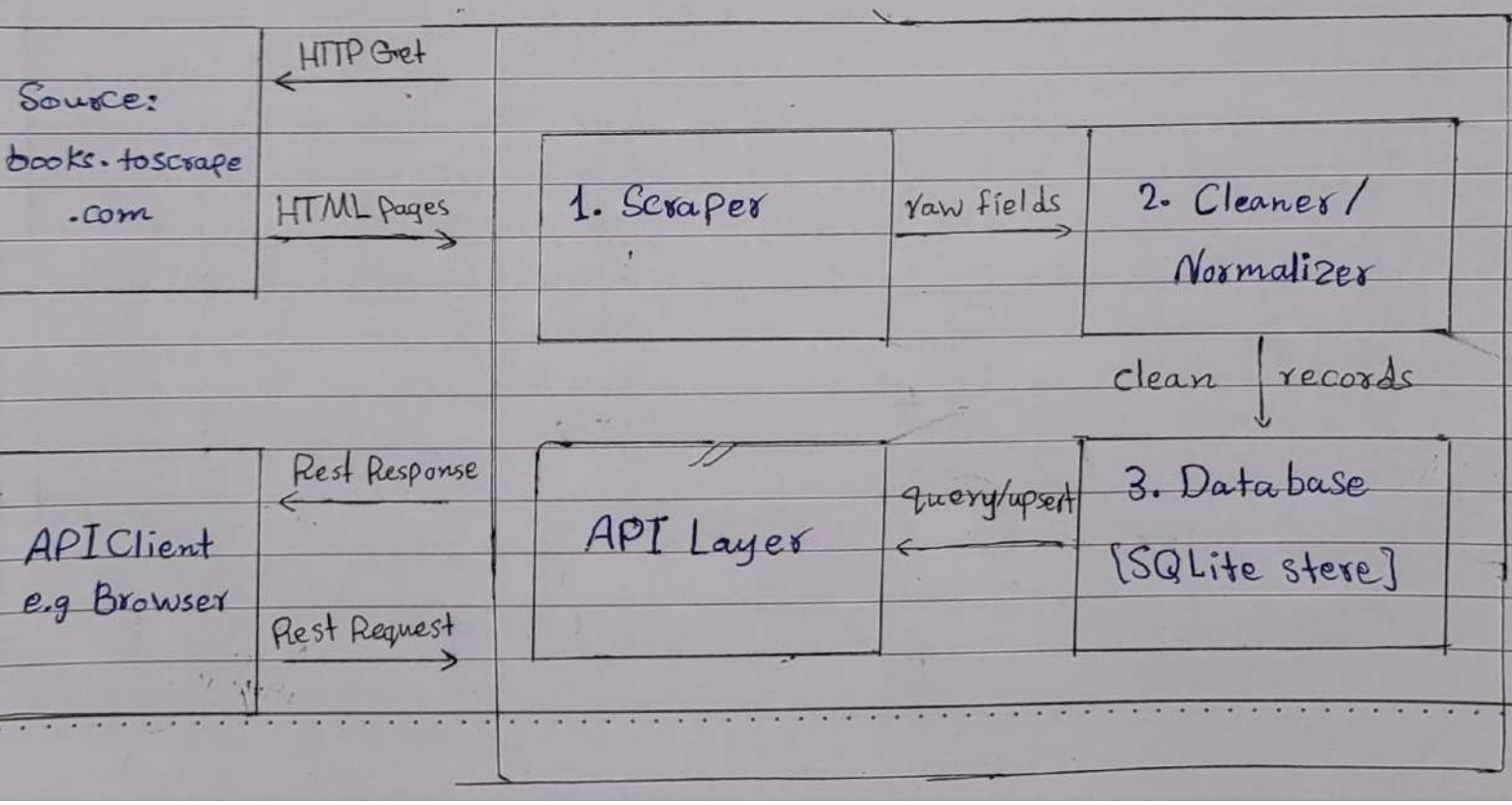
- The data pipeline follows a clear sequence:

Website → Scraper → Cleaner → Database → API → Client

* scraper.py: extracts raw book fields from HTML pages.
* cleaner.py: converts raw text into normalized, typed values.
* database.py: stores final records in SQLite with stable schema rules.
* api.py: serves data to external consumers via REST endpoints.

- A secondary control path supports data refresh:

Client → /scrape/trigger → pipeline.py → scrape + clean + upsert



Date No.

Data Cleaning Logic:

- Price normalization: text values ⟶ numeric float values
- Rating mapping: text lables (One, Two...) ⟶ integers (1-5)
- Aailability normalization: stock messages ⟶ predictable status values.
- Deterministic IDs: stable IDs are derived from URLs to preserve identity across repeated runs.
- Deduplication: records are keyed by product_url to prevent duplicates on re-scrape.

File Organization Rationale:

The Project is organized by operational responsibility:

* src/scraping_service/: Core application logic
* scripts/: helper scripts for local execution and dev tasks.
* deploy/: runtime infrastucture assets (systemd unit, firewall setup...)
* Packaging/: Debian packaging scripts and hooks.
* docs/: design notes, analysis and other docs.

Date No.

API Design Choices:

Endpoint design is grouped by purpose:
- Service Health: /health
- Data Retrieval: /books, /books/{id}
- Analytics views: /stats/categories, /stats/ratings
- Operational Control: /scrape/trigger

* Listing endpoint /books supports pagination via: page and page_size parameters, and filtering via: category and min_rating parameters.

Deployment and Service Lifecycle:

The API runs under Systemd to ensure robust process management:
* stars automatically on boot.
* restarts on failure.
* runs under a dedicated service account.
* loads config from an env file.

Date No.

Packaging Flow:

- The Debian packaging process assembles the app into an installable artifact using build_deb.sh script. Installation places files in standard Linux paths such as:

  * application files: /opt
  * executable wrappers: /usr/bin
  * Configuration in: /etc
  * Service unit in: /lib/systemd/system

- Maintainer scripts automate lifecycle actions:

  * postinst: Create service user/dirs and enable/starts the service.
  * prerm / postrm: handle stutdown and cleanup during removal.

- This gives users a one-command install/remove experience.